

ChartNet: A Million-Scale, High-Quality Multimodal Dataset for Robust Chart Understanding

Jovana Kondic^{1*}, Pengyuan Li³, Dhiraj Joshi³, Isaac Sanchez², Ben Wiesel³, Shafiq Abedin³, Amit Alfassy³, Eli Schwartz³, Daniel Caraballo³, Yagmur Gizem Cinar³, Florian Scheidegger³, Steven I. Ross³, Daniel Karl I. Weidele², Hang Hua², Ekaterina Arutyunova¹, Roei Herzig³, Zexue He², Zihan Wang⁴, Xinyue Yu⁴, Yunfei Zhao⁴, Sicong Jiang⁴, Minghao Liu⁴, Qunshu Lin⁴, Peter Staar³, Luis Lastras³, Aude Oliva^{1,2}, Rogerio Feris^{2,3†}

¹MIT ²MIT-IBM Watson AI Lab ³IBM Research ⁴Abaka AI

Abstract

Understanding charts requires models to jointly reason over geometric visual patterns, structured numerical data, and natural language — a capability where current vision-language models (VLMs) remain limited. We introduce ChartNet, a high-quality, million-scale multimodal dataset designed to advance chart interpretation and reasoning. ChartNet leverages a novel code-guided synthesis pipeline to generate 1.5 million diverse chart samples spanning 24 chart types and 6 plotting libraries. Each sample consists of five aligned components: plotting code, rendered chart image, data table, natural language summary, and question-answering with reasoning, providing fine-grained cross-modal alignment. To capture the full spectrum of chart comprehension, ChartNet additionally includes specialized subsets encompassing human annotated data, real-world data, safety, and grounding. Moreover, a rigorous quality-filtering pipeline ensures visual fidelity, semantic accuracy, and diversity across chart representations. Fine-tuning on ChartNet consistently improves results across benchmarks, demonstrating its utility as large-scale supervision for multimodal models. As the largest open-source dataset of its kind, ChartNet aims to support the development of foundation models with robust and generalizable capabilities for data visualization understanding. The dataset is publicly available at <https://huggingface.co/datasets/ibm-granite/ChartNet>.

1. Introduction

Charts are a fundamental medium for communicating quantitative information across scientific, financial, and business

domains. They translate structured data into visual form, allowing readers to efficiently reason about trends, distributions, and relationships. However, interpreting such visualizations requires integration of visual, numerical, and linguistic understanding — a capability that current vision-language models (VLMs) only partially achieve.

Despite a growing body of work on chart understanding and reasoning, progress remains bounded by data limitations. Existing datasets are often limited in size, narrow in scope, or incomplete in their multimodal coverage. Many focus on a single task (e.g., question answering or captioning) or lack critical modalities such as plotting code, grounding annotations, or reasoning traces. Consequently, open-source models continue to lag behind proprietary systems in complex chart reasoning tasks that demand tight coupling between visual perception, structured data extraction, and natural language interpretation.

To address this gap, we introduce *ChartNet*, a million-scale, high-quality multimodal dataset designed to advance robust chart understanding. ChartNet builds on a code-guided synthetic generation pipeline capable of producing chart tuples at scale that jointly capture the visual, structural, numerical, and textual aspects of chart understanding. Each instance in the dataset includes a rendered chart image, executable plotting code, underlying data table, natural-language summary, and question-answering with reasoning, ensuring complete modality alignment and interpretability. In addition, ChartNet incorporates real-world and human-annotated data, as well as specialized subsets supporting grounding, and safety analysis — broadening the dataset’s utility for both model training and evaluation.

We perform a thorough experimental analysis, and demonstrate the value of ChartNet across models of various sizes on multiple chart understanding tasks. We also find that our best finetuned model outperforms models order-of-magnitude larger as well as GPT-4o across all tasks.

*Correspondence: jkondic@mit.edu

†Correspondence: rsferis@us.ibm.com

Dataset	#Charts	#Types	#Plot Libs	Real	Code	Data	Text	QA	Reason.	Human	Grounding	Safety	Task
FigureQA [22]	100K	3	1	✗	✗	✗	✗	✓	✗	✗	✗	✗	Binary QA
DVQA [21]	300K	1	1	✗	✗	✗	✗	✓	✗	✗	✗	✗	Fixed QA
PlotQA [42]	224K	4	1	✓	✗	✗	✗	✓	✗	✗	✗	✗	Open QA
ChartQA [36]	14K	3	1	✓	✗	✗	✗	✓	✗	✓	✗	✗	Complex QA
Chart-to-Text [24]	44K	Multi	1	✓	✗	✗	✓	✗	✗	✓	✗	✗	Summary
OpenCQA [23]	7.7K	Multi	1	✓	✗	✓	✓	✓	✗	✓	✗	✗	Long QA
UniChart [37]	611K	3	Multi	✓	✓	✓	✓	✓	✗	✓	✗	✗	Multi-task
ChartLlama [9]	7.8K	10+	1	✗	✓	✗	✓	✓	✗	✗	✗	✗	Multi-task
StructChart [58]	9K	3	1	✗	✓	✓	✗	✗	✗	✗	✗	✗	Structure
MMC [29]	600K	6	Multi	✓	✗	✓	✓	✓	✗	✓	✗	✗	Instruct
ChartX [59]	6K	18	4+	✗	✓	✓	✓	✓	✗	✓	✗	✗	Multi-task
TinyChart [64]	680K	Multi	Multi	✓	✗	✗	✓	✓	✓	✓	✗	✗	QA+Summary
ChartQAPro [39]	1.3K	Multi	1	✓	✗	✗	✗	✓	✓	✗	✗	✗	Diverse QA
Plot2Code [57]	132	6	1	✗	✓	✗	✗	✗	✓	✗	✗	✗	Code Gen
ChartMimic [62]	4.8K	22	Multi	✓	✓	✗	✗	✗	✗	✓	✗	✗	Chart-to-code
ChartCoder [66]	160K	27	1	✗	✓	✗	✗	✗	✗	✗	✗	✗	Code Gen
CoSyn [63]	118K	22	3+	✗	✓	✓	✓	✓	✓	✗	✗	✗	Multi-task
ChartNet (ours)	1.5M	24	6	✓	✓	✓	✓	✓	✓	✓	✓	✓	Multi-task

Table 1. Comparison of chart understanding datasets. We report the number of unique chart images, chart types, and plotting libraries included; types of data modalities included—real-world charts/data, plotting code, tabular data, text descriptions, question-answer pairs, reasoning traces, human annotations, grounding signals, and safety data—and the scope of chart understanding tasks covered.

Our contributions are threefold:

1. We propose a code-guided automatic chart generation pipeline that integrates structured data synthesis with automated quality filtering, ensuring visual fidelity, semantic correctness, and representational diversity at scale.
2. We release *ChartNet*, the largest-to-date synthetic chart dataset, spanning diverse chart types, plotting libraries, and topics. It contains 1.5 million high-quality multi-modal tuples (image, code, CSV, text, and reasoning-based QA), as well as subsets including human annotations, grounding, safety data, and real-world charts.
3. We demonstrate the utility of ChartNet through comprehensive experiments, showing that finetuning on this dataset consistently improves chart reconstruction, data extraction, and chart summarization performance across vision–language models.

ChartNet establishes a new standard for multimodal chart understanding by unifying scale, diversity, and representational completeness, enabling the next generation of models to reason over data visualizations with greater accuracy and generalization.

2. Related Work

2.1. Large Multimodal Models.

Open-source multimodal models [2, 10, 30, 52, 53] have made notable progress on document and chart comprehension benchmarks, yet their performance generally falls short of leading proprietary models. Recent efforts to close this

gap include architectural improvements, such as enhanced high-resolution image processing [8, 13, 67] and explicit numerical reasoning [48, 64]. Nevertheless, the scarcity of high-quality chart comprehension training data remains a critical bottleneck. This challenge is compounded by the lack of transparency surrounding data curation practices in even the best-performing open models [41, 53], creating significant barriers to reproducibility. Our ChartNet dataset, on the other hand, provides large scale, high quality data for advancing the chart understanding capabilities of multimodal models, while being made freely available to the research community.

2.2. Chart Understanding Datasets

Numerous datasets have been proposed for chart question-answering [17, 21–23, 36, 42, 43], captioning and summary generation [24, 49], chart-to-code translation [45, 57, 62, 66], and multimodal chart reasoning [9, 31, 40, 44, 55, 59, 61, 63]. However, these datasets fail to capture the full diversity of real-world charts. For example, ChartQA [36] – a widely used benchmark for multimodal models – encompasses only a few chart types (bar, line, and pie charts) obtained from limited online sources. Moreover, it is biased towards questions requiring basic data extraction, resulting in performance saturation for modern vision-language models. While recent datasets have addressed some of these limitations by incorporating more realistic charts [27] and more complex questions [39], they still lack the diversity, scale, and quality required to train frontier large multimodal mod-

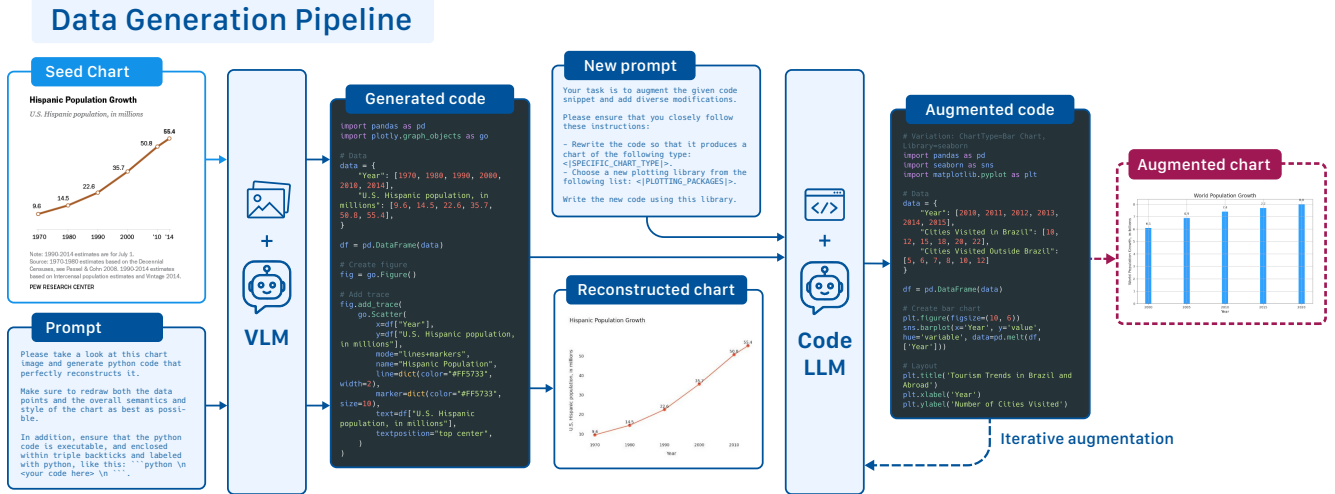


Figure 1. Code-guided chart augmentation: First, a seed chart image is passed to a vision-language model for chart reconstruction – translating the image into executable plotting code. Then, the generated code is passed to a large language model, and iteratively augmented to collect diverse outputs.

els. In contrast, ChartNet is a million-scale dataset featuring 24 different chart types and various plotting libraries, with rigorous data filtering, high-quality human annotations, and associated tasks including chart-to-code, chart data extraction, chart captioning, reasoning, grounding, and safety. Table 1 compares ChartNet with other datasets.

2.3. Synthetic Data Generation for Vision-Language Models

Recently, synthetic data generation has gained significant attention from both industry and academia as an effective means to improve the capabilities of VLMs [7, 14, 68]. It has proven especially valuable for tasks such as visual question answering [3, 15, 25, 35] and compositional reasoning [11, 12, 19, 20, 50]. In contrast, our approach performs data generation and augmentation in the code space as opposed to the image space. Granite Vision [51], DAVE [16], SmolDocling [32], Molmo [6], and CoSyn [63] also rely on synthetic data generation for charts and documents tasks. Different from our work, they exhibit limited diversity in chart types and modalities compared to ChartNet.

3. ChartNet Data Generation Pipeline

A key methodological insight underlying our data generation is that charts are generated programmatically: executable plotting code serves a structured intermediate representation for data visualizations [26]. We introduce an automated pipeline for code-guided synthetic chart generation at scale (see Figure 1). Starting with a limited dataset of chart images (“seeds”), a VLM outputs code that approximately reconstructs them. We then leverage the code representation to (1) iteratively generate augmentations, producing vi-

sually and semantically diverse charts, and (2) generate additional semantic attributes, including tabular data, natural language descriptions, and question-answering traces with chain-of-thought reasoning.

3.1. Code-Guided Data Generation At Scale

Specifically, our data generation pipeline consists of the following stages:

- 1. Chart-to-Code Reconstruction:** We prompt a VLM to produce Python plotting code that approximately reconstructs a given set of chart images. Here, we select a seed set of 150,000 unique chart images from TinyChart [64], though the pipeline is agnostic to this choice.
- 2. Code-Guided Chart Augmentation:** Using the produced plotting code as input, we prompt an LLM to iteratively rewrite it. The underlying data values and labels are transformed to better match the requested chart type, while maintaining relevance to the previous iteration. Figure 2 illustrates the iterative code augmentation and chart rendering process. This stage is the primary contributor to dataset scaling, taking each seed image and producing up to an arbitrary number of variations.
- 3. Chart Rendering:** We execute all the generated plotting code to produce chart images. The scripts that were successful upon execution are paired with the image that they produced.
- 4. Quality Filtering:** Using a VLM, we evaluate each chart image across multiple identified categories of potential rendering defects (e.g., overlapping text, cropped labels, obscured features). Images classified with visual issues (and their plotting code) are removed.
- 5. Code-Guided Attribute Generation:** Finally, we use a

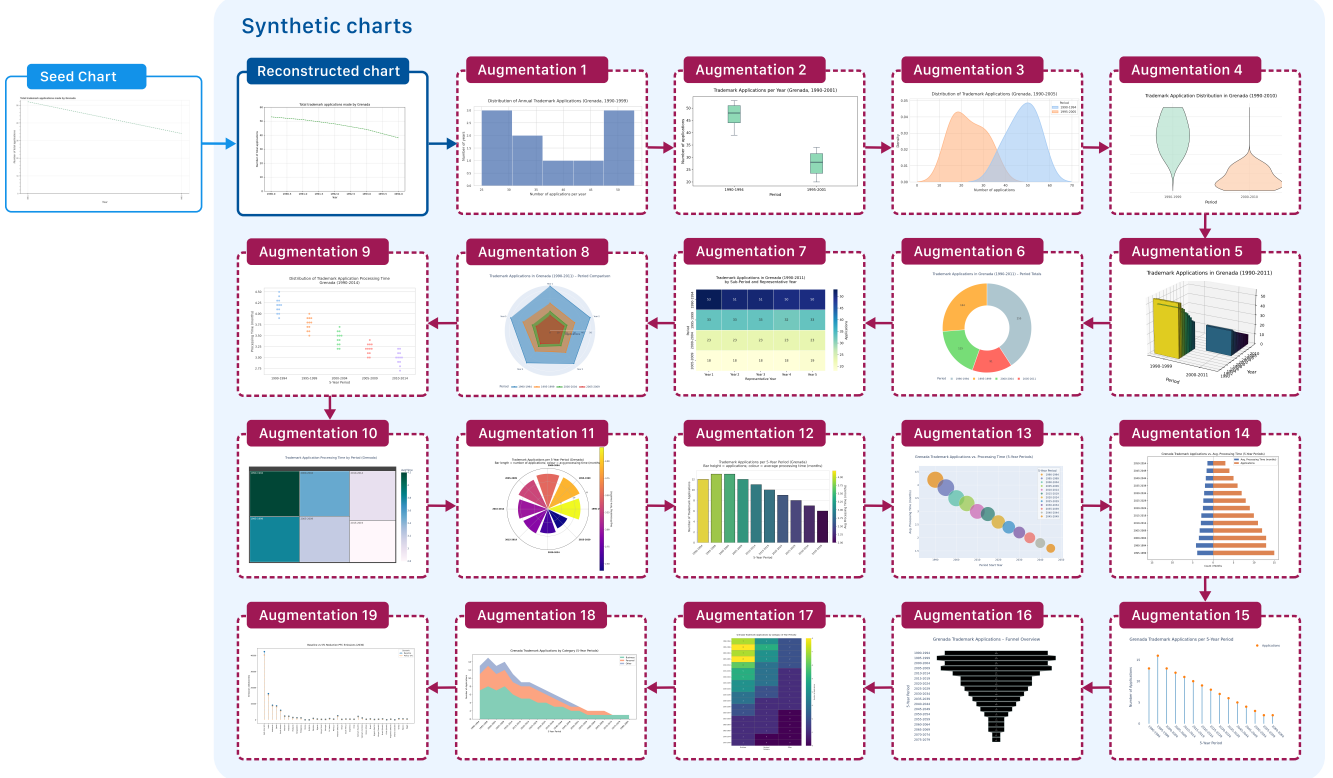


Figure 2. An illustration of synthetic chart images generated from a single seed chart using the ChartNet pipeline. A seed chart is first translated into approximate plotting code, which is executed to render a reconstructed chart. The code is then iteratively augmented to produce diverse variations in chart types, styles, and representations, as shown in the subsequent *augmentations*.

VLM to generate supplementary semantic attributes to the chart image-code pairs. Grounding the visual information with code as additional context, we extract the data values and labels from charts and produce tabular data representations. Furthermore, combining the visual context with code and tabular data, we produce grounded chart descriptions.

For prompt templates used, see Section B.1.

3.2. QA Pairs with CoT Reasoning

In addition to chart image, code, tabular data, and natural language descriptions, we also generate question-answer (QA) pairs with long Chain-of-Thought (CoT) reasoning as part of the ChartNet dataset. This data generation process is built on the Vision-R1 framework [18]. Using pixtral-large-instruct-2411, we generate a complex multi-stage reasoning question for each image in the ChartNet dataset. Next, following the procedure proposed in LLaVA-CoT [60], we construct a four-step “Pseudo-CoT” sequence (Summary, Caption, Reasoning, and Conclusion) using separate model calls. We then perform modality bridging, where the model describes the complete visual content in relation to the Pseudo-CoT, enabling a language-only model to reason ef-

fectively without direct visual input. Finally, gpt-oss-120b [1] produces detailed textual reasoning traces and final predictions enclosed within `<think>` and `<answer>` tags. This multi-stage pipeline produces rich, verifiable reasoning traces while preserving strong alignment between visual and textual representations. See Section A.2 for more information and illustrative examples.

3.3. Models and Compute Infrastructure

Our model choice was based on a combination of demonstrated performance and adhering to open-source values. We use pixtral-large-instruct-2411 in the Chart-to-Code Reconstruction, Quality Filtering, and Code-Guided Attribute Generation stages, and gpt-oss-120b in the Code-Guided Chart Augmentation stage. For scale, we deployed multiple replicas of both models on over a hundred A100 and H100 GPUs. The work was distributed across the GPUs to maintain high throughput, generating over 1 million annotated data points roughly every 168 hours.

3.4. Quality Filtering Evaluation

In the Quality Filtering stage, we track three observable metrics across three stages, and observe the following:

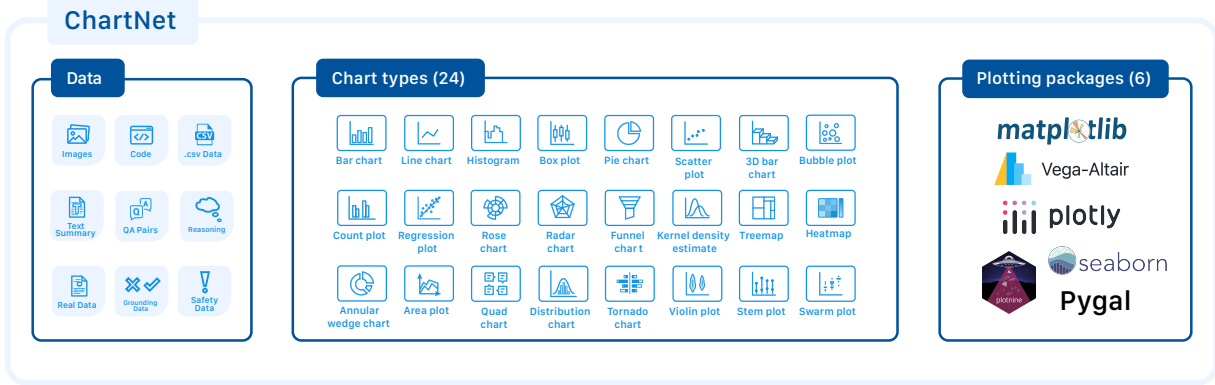


Figure 3. Data attributes, chart types, and plotting packages included in ChartNet.

- **Probability of Failure** (Chart Augmentation): The model fails to rewrite the code snippet with requested changes and proper formatting in $<0.01\%$ of requests.
- **Execution Rate** (Chart Rendering): On average, 77% of the generated code snippets execute successfully.
- **Visual Error Rate** (Quality Filtering): On average, 36.5% of rendered images were classified to contain some visual error.

To quantify how well pixtral-large-instruct-2411 aligns with human performance in detecting visual defects, 3157 randomly sampled charts were manually annotated and compared to the corresponding model prediction. Before Quality Filtering, 14.9% of generated samples were found to contain issues that affect chart readability. After Quality Filtering, only 5.9% of the charts contained these issues.

4. The ChartNet Dataset

4.1. Core Dataset

The core ChartNet dataset consists of 1.5M multimodal aligned synthetic tuples: chart image, plotting code, tabular data, natural language description, and QA pairs with CoT reasoning. For a complete overview of the data attributes, chart types, and plotting packages included, see Fig. 3.

To capture the full spectrum of chart understanding, ChartNet additionally includes specialized subsets: human-annotated data, real-world charts, grounding, and safety.

4.2. Human-Annotated Synthetic Chart Data

In addition to the core dataset, we curate a high-quality subset of 96,643 aligned synthetic chart images, descriptions, and tabular data that have gone through rigorous human verification and annotation. See Section A.3 for more information about the annotation process.

4.3. High-Quality Real-World Charts

To complement our synthetic chart corpus, we also curate and annotate 30K real-world charts sourced from reputable

international media and data-visualization outlets such as the *World Bank* [56], *Bain Insights* [5], *Pew Research Center* [47], *Our World in Data* [46], and other globally recognized publishers. This collection captures a broad spectrum of contemporary topics, including economics, technology, geopolitics, environmental science, and societal trends, also ensuring high diversity and strong real-world relevance. We explicitly discard a broad set of low-information or low-quality visuals that do not meet our interpretability standard. To ensure full compliance with copyright and data-use regulations, all real-world charts were collected exclusively from legally safe, openly licensed, or public-domain sources, and their use falls strictly under non-commercial academic research exceptions.

Each selected chart is paired with metadata, including its caption, sub-caption, key data highlights, and a concise analytical summary, to support joint learning of visual reasoning, textual grounding, and high-level insight extraction. This subset is specifically designed to strengthen multimodal model performance on challenging chart understanding tasks, including:

- **Quantitative and comparative reasoning**: extracting values, trends, anomalies, and multi-series comparisons directly from visual structures;
- **Chart-text semantic alignment**: linking visual elements with captions, labels, and narrative descriptions;
- **Context-aware summarization**: generating coherent explanations that integrate both visual evidence and accompanying textual information;
- **Cross-lingual interpretation**: supporting multilingual understanding of globally sourced visualizations.

For additional information and illustrative examples, see Section A.4.

4.4. Grounding QA Pairs

Modern VLMs still struggle to identify the chart areas and syntactic elements relevant to a given question. To further advance such capabilities, we create grounding QA pairs.

First, we extract geometry-aware annotations from elements of the plotting code (axes, ticks, gridlines, legends, patches) to produce dense grounding annotations of the corresponding charts. Bounding boxes are further filtered using an entropy-based approach (see Section A.5.1). Using the resulting grounded annotations, for each chart, we create a set of template-based QAs that capture the duality between the expected spatial arrangement of visual elements and the observed content depicted in the plots. The expected locations are encoded as serialized bounding-boxes representations within the corresponding answer strings.

Templates address unique and recurring visual elements, incorporating referring expressions based on indices, textual labels present in the plot, and visual attributes (e.g., element color). The generator supports both short- and long-form answers, and can optionally include grounding information for each. The final dataset is obtained by uniform sampling across all template types and output modalities, generating one QA pair per image. In addition to this, we include a set of reasoning-based grounding QA pairs by leveraging gpt-oss-120b. Section A.5 provides more information and points to examples of the generated QA pairs.

4.5. Safety

To address safety concerns, we extend our pipeline to generate chart-related safety alignment data that mitigates harmful model outputs and jailbreak vulnerabilities. We first select charts with sensitive content across topics including health, finance, and social issues. We then synthetically generate adversarial questions spanning categories such as discrimination, hate, violence, political bias, and substance abuse (e.g., "Does this bar chart prove that Race X causes higher crime rates?"). Each question is paired with both safe and unsafe responses, creating preference pairs suitable for direct preference optimization. We release 7,000 training samples and 600 test samples as part of ChartNet. For prompt templates and more information, see Section A.6.

5. Experiments

5.1. Model Training

We train VLMs of various sizes on the ChartNet dataset to validate its effectiveness in enhancing models' chart understanding capabilities. The supervised finetuning (SFT) data comprises the four tasks of the core ChartNet dataset: **Chart-to-Code**, **Chart-to-Table**, **Chart-to-Text**, and **Chart QA with CoT Reasoning**. Specifically, we experiment with different model scales: Ultra-Compact ($\leq 1B$) — Granite-Docling-258M [33] and SmolVLM-256M-[34]; Small ($\leq 4B$) — Granite-vision-3.3-2b [51] and Qwen2.5-VL-3B-Instruct [4]; and Medium ($\leq 7B$) — LLaVA-v1.6-mistral-7b [30]. We follow the default hyperparameter settings provided by the TRL[54] codebase.

5.2. ChartNet Evaluation Set

To rigorously evaluate the tasks in the core ChartNet dataset, we curate a held-out evaluation suite randomly drawn from ChartNet's synthetic corpus. The set comprises 2000 chart tuples, each including a chart image, its corresponding plotting code, underlying data table, a natural language summary, and QA pairs with CoT reasoning. We evaluate model performance across four tasks:

Chart Reconstruction (Chart-to-Code). Given a chart image I , the model is required to generate an executable plotting script C' that reproduces as closely as possible the source code C used to render the input chart I . We evaluate (a) *execution rate* (Exec.) — the fraction of generated scripts C' that execute without error, (b) *data fidelity* (Code-D) — the correspondence between plotted numeric values and the data defined in ground-truth code, (c) *code similarity* (Code-S) — the structural and syntactic overlap between generated, C' , and source code, C , and (d) *rendered image similarity* (Img.) — the visual alignment between the rendered prediction and the input chart I .

Chart Data Extraction (Chart-to-Table). This task evaluates the ability of a model to infer the plotted data directly from the chart image. Given an input image I , a model is asked to produce a CSV table that matches as closely as possible the data points visualized in I . Using I as context, we compare the generated data table to the ground-truth CSV, and report a similarity score disregarding minor formatting differences.

Chart Summarization (Chart-to-Text). Given a chart image I , the model is tasked with generating a comprehensive textual summary capturing the key takeaways, data trends, comparisons, and visual elements and style of the chart. Using I as context, we compare the generated summary to the reference summary generated and verified by the ChartNet data generation pipeline as described in Section 3. We report a holistic score encompassing the coverage of key elements, faithfulness to the visual, semantic and numeric correctness, and clarity.

Chart QA with CoT Reasoning For each chart image I , we pair the generated complex reasoning question with I , and prompt the model to output `<think>` and `<answer>` sections. The final answer is extracted from `<answer>` and compared to the gold reference using RapidFuzz for fuzzy string matching. We report average fuzzy accuracy.

¹Latest updates and enhancements to the dataset are available at <https://huggingface.co/datasets/ibm-granite/ChartNet>. See also results using granite-4.0-3b-vision finetuned on ChartNet at <https://huggingface.co/ibm-granite/granite-4.0-3b-vision>.

Model (params)	Chart Reconstruction				Chart Data Extraction	Chart Summarization	Chart QA w/ CoT Reasoning
	Exec.	Code-D	Code-S	Img.			
SmolVLM-Instruct (256 M)	N/A	N/A	N/A	N/A	22.0	26.6	55.0
+ ChartNet	14.9	56.8	74.9	77.5	36.4	60.0	60.8
(+)	(+14.9)	(+56.8)	(+74.9)	(+77.5)	(+14.4)	(+33.4)	(+5.8)
granite-docling-258M (258 M)	N/A	N/A	N/A	N/A	17.7	24.1	54.0
+ ChartNet	41.8	49.7	63.1	70.4	32.0	55.5	61.0
(+)	(+41.8)	(+49.7)	(+63.1)	(+70.4)	(+14.3)	(+31.4)	(+7.0)
granite-vision-3.3-2B (2 B)	63.4	60.7	67.0	77.2	53.8	64.0	59.9
+ ChartNet	90.4	72.8	90.0	92.8	70.3	83.9	65.0
(+)	(+27.0)	(+12.1)	(+23.0)	(+15.6)	(+16.5)	(+19.9)	(+5.1)
Qwen2.5-VL-3B-Instruct (3 B)	65.1	52.6	68.0	76.7	51.8	70.6	58.4
+ ChartNet	74.1	59.1	76.1	82.8	62.4	80.1	69.9
(+)	(+9.0)	(+7.5)	(+8.1)	(+6.1)	(+10.6)	(+9.5)	(+11.5)
llava-vl.6-mistral-7b-hf (7 B)	45.3	27.0	52.9	59.6	17.0	51.2	55.1
+ ChartNet	83.9	69.4	88.6	91.5	58.8	80.3	70.3
(+)	(+38.6)	(+42.4)	(+35.7)	(+31.9)	(+41.8)	(+29.1)	(+15.2)

Table 2. Paired comparison of base models vs finetuned models on the ChartNet evaluation set, with performance gains in blue ¹. Each model variant was trained solely on the subset of the ChartNet dataset corresponding to the specific task it was evaluated on (for example, models evaluated on Chart Reconstruction were trained only on the Chart-to-Code subset of ChartNet).

Model (params)	Chart Reconstruction				Chart Data Extraction	Chart Summarization	Chart QA w/ CoT Reasoning
	Exec.	Code-D	Code-S	Img.			
Open-Source Models							
Qwen3-VL-3B-Instruct (3 B)	76.9	63.3	74.3	86.8	58.1	79.2	64.3
InternVL3.5-8B (8 B)	69.1	60.2	69.9	78.0	56.1	71.3	61.6
Pixtral-12B-2409 (12 B)	74.9	52.9	72.9	81.4	49.1	77.5	60.0
Mistral-Small-3.1-24B-Instruct-2503 (24 B)	88.1	54.5	74.8	86.3	53.2	79.8	60.0
Qwen2-VL-72B-Instruct (72 B)	83.1	50.7	67.3	77.6	50.3	75.9	60.3
Chart Models							
ahmed-masry/chartgemma (3B)	N/A				37.1	46.1	69.5
Proprietary Models							
GPT-4o	95.9	48.8	77.2	88.2	46.7	77.1	61.1

Table 3. Performance of off-the-shelf models on the ChartNet evaluation set.

We evaluate a range of off-the-shelf open-source VLMs (< 1B – 72B parameters), a specialized chart model (ChartGemma [38]), and GPT-4o, and compare these against models finetuned on ChartNet (as outlined in Section 5.1). All metrics are automatically computed using GPT-4o as a judge, except for the Chart QA with CoT Reasoning task. The prompt templates used are listed in Section B.4.

5.3. Public Benchmarks

We additionally evaluate ChartNet on established public benchmarks including chart summarization (ChartCap [28]) and chart-to-code generation (ChartMimic-v2 [62]). We follow the original evaluation protocols and report standard metrics, comparing both off-the-shelf models and their ChartNet-finetuned variants to prior open-source baselines.

6. Results & Discussion

As shown in Table 2, finetuning on ChartNet produces substantial and consistent gains across all chart understanding tasks. The uniformity and magnitude of these improvements – regardless of model scale – indicate that existing VLMs lack exposure to high quality multimodal chart supervision, and ChartNet fills this gap effectively.

Chart Reconstruction Models trained on the Chart-to-Code subset show large improvements in code execution rates, data fidelity, and structural/code and image similarity. Ultra compact models that originally could not reconstruct charts at all (SmolVLM-256M, Granite-Docling-258M) gain fully functional capability, while small models such as Granite-Vision-2B achieve near-perfect reconstruction, reaching 90%+ on most metrics. The LLaVA-7B model experiences gains of up to +42.4 points, substantially

Model	ChartCap (Summarization)			ChartMimic-v2 (Code Generation)	
	BLEU.4	METEOR	ROUGE.L	v2-direct	v2-customized
SmolVLM-256M-Instruct	0.60	14.30	13.50	0	0
SmolVLM-256M-Instruct-ChartNet	3.00	22.00	17.80	12.67	11.67
granite-vision-3.3-2b	1.60	6.40	9.60	30.84	30.45
granite-vision-3.3-2b-ChartNet	12.40	30.10	24.90	58.42	51.21
llava-next-mistral-7b	6.40	22.60	20.90	28.20	30.76
llava-next-mistral-7b-ChartNet	7.10	27.00	22.10	54.78	38.27

Table 4. Generalizability of gains from ChartNet synthetic data on two real-world public benchmarks.

improving the code data fidelity performance. The scale-invariant trend suggests that ChartNet’s multimodal alignment between images and code provides a type of structural supervision unavailable in prior datasets.

Chart Data Extraction ChartNet dramatically boosts all models’ ability to recover numeric tables directly from chart images, with the best performing Granite-Vision-2B scoring 70.3%. The finetuned LLaVA-7B model improves performance by +41.8, exceeding every open-source baseline (including those in Table 3) and surpassing GPT-4o which shows particularly limited performance in this task, at only 46.7% accuracy. This reflects the value of ChartNet’s tight coupling between the code-generated charts and CSVs, which gives models explicit exposure to both visual geometry and underlying numeric structure.

Chart Summarization Summarization quality improves across all model families, with gains ranging from +9.5 (Qwen2.5-VL-3B) to +31.4 (Granite-Docling-2B). The finetuned Granite-Vision-2B reaches 83.9%, surpassing GPT-4o and all open-source baselines in Table 3 including those that are an order of magnitude larger. This suggests that ChartNet’s synthetic summaries, constructed jointly from code and rendered visuals, provide precisely the kind of structured, semantically complete supervision needed for descriptive chart understanding.

QA with CoT Reasoning Every model exhibits steady accuracy improvements on the complex multi-stage reasoning task. LLaVA-7B achieves the largest improvement (+15.17), reaching 70.3%, outperforming a specialized chart reasoning model (ChartGemma) and all other baselines of comparable or order-of-magnitude larger sizes (including GPT-4o) in Table 3.

Comparison with Off-the-Shelf Models Table 3 highlights that ChartNet-tuned models outperform far larger off-the-shelf models in nearly every metric. A 2B or 7B model finetuned on ChartNet consistently exceeds the performance of 20B–72B models trained on conventional multimodal corpora. In Chart Reconstruction and Chart Data Extraction, the gap is especially pronounced: ChartNet-tuned models far surpass GPT-4o overall. These results

point toward an emerging principle: for domains like chart interpretation, where visual, numerical, and linguistic information are tightly coupled, scaling model size is far less effective than providing high-quality, code-aligned multimodal supervision.

Collectively, these findings show the utility of ChartNet in boosting the capabilities of VLMs, enabling robust, interpretable, and numerically grounded chart reasoning that is otherwise unreachable with vision–language training.

Generalization to Public Benchmarks As shown in Table 4, finetuning on the core ChartNet dataset (Section 4.1) yields substantial absolute gains across all models. Notably, Granite-Vision-2B improves from 1.6 to 12.4 BLEU on ChartCap, and from 30.8 to 58.4 on ChartMimic-v2, and even ultra-compact models (SmolVLM-256M) gain non-trivial capability. These improvements are consistent across both chart summarization and chart-to-code translation tasks, indicating that the aligned multimodal supervision of ChartNet transfers effectively to real-world benchmarks beyond the synthetic training distribution.

7. Conclusion

We present ChartNet, addressing a central bottleneck in chart understanding: the lack of large-scale, high-fidelity supervision that aligns images, plotting code, numeric data, textual descriptions, and reasoning traces. By generating over one million aligned multimodal tuples, ChartNet equips VLMs with programmatically grounded knowledge that transfers across chart-to-code reconstruction, data extraction, summarization, and multi-step reasoning. Experiments show consistent gains across model sizes and architectures, often surpassing much larger open-source systems and even proprietary frontier models such as GPT-4o. These gains are not limited to any single task; they indicate a broader improvement in how models internalize chart semantics when trained with code-grounded supervision. ChartNet offers a scalable, open foundation for research in numerical reasoning, visualization understanding, document intelligence, and code-aligned multimodal modeling—moving VLMs from describing charts toward understanding the structured information they encode.

Acknowledgments

This work was supported in part by funding from the MIT-IBM Watson AI Lab.

References

- [1] Sandhini Agarwal, Lama Ahmad, Jason Ai, Sam Altman, Andy Applebaum, Edwin Arbus, Rahul K Arora, Yu Bai, Bowen Baker, Haiming Bao, et al. gpt-oss-120b & gpt-oss-20b model card. *arXiv preprint arXiv:2508.10925*, 2025. 4, I, II
- [2] Xiang An, Yin Xie, Kaicheng Yang, Wenkang Zhang, Xiuwei Zhao, Zheng Cheng, Yirui Wang, Songcen Xu, Changrui Chen, Chunsheng Wu, et al. Llava-onevision-1.5: Fully open framework for democratized multimodal training. *arXiv preprint arXiv:2509.23661*, 2025. 2
- [3] Stanislaw Antol, Aishwarya Agrawal, Jiasen Lu, Margaret Mitchell, Dhruv Batra, C Lawrence Zitnick, and Devi Parikh. Vqa: Visual question answering. In *Proceedings of the IEEE international conference on computer vision*, 2015. 3
- [4] Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibao Song, Kai Dang, Peng Wang, Shijie Wang, Jun Tang, et al. Qwen2. 5-vl technical report. *arXiv preprint arXiv:2502.13923*, 2025. 6
- [5] Bain. Bain & company insights. <https://www.bain.com/insights/>, 2025. 5
- [6] Matt Deitke, Christopher Clark, Sangho Lee, Rohun Tripathi, Yue Yang, Jae Sung Park, Mohammadreza Salehi, Niklas Muennighoff, Kyle Lo, Luca Soldaini, et al. Molmo and pixmo: Open weights and open data for state-of-the-art multimodal models. *arXiv e-prints*, 2024. 3
- [7] Jiawei Guo, Tianyu Zheng, Yizhi Li, Yuelin Bai, Bo Li, Yubo Wang, King Zhu, Graham Neubig, Wenhui Chen, and Xiang Yue. Mammoth-vl: Eliciting multimodal reasoning with instruction tuning at scale. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*, 2025. 3
- [8] Zonghao Guo, Ruyi Xu, Yuan Yao, Junbo Cui, Zanlin Ni, Chunjiang Ge, Tat-Seng Chua, Zhiyuan Liu, and Gao Huang. Llava-uhd: an lmm perceiving any aspect ratio and high-resolution images. In *European Conference on Computer Vision*, 2024. 2
- [9] Yucheng Han, Chi Zhang, Xin Chen, Xu Yang, Zhibin Wang, Gang Yu, Bin Fu, and Hanwang Zhang. Chartllama: A multimodal llm for chart understanding and generation, 2023. 2
- [10] Wenyi Hong, Wenmeng Yu, Xiaotao Gu, Guo Wang, Guobing Gan, Haomiao Tang, Jiale Cheng, Ji Qi, Junhui Ji, Li-hang Pan, et al. Glm-4.1 v-thinking: Towards versatile multimodal reasoning with scalable reinforcement learning. *arXiv e-prints*, 2025. 2
- [11] Hang Hua, Jing Shi, Kushal Kafle, Simon Jenni, Daoan Zhang, John Collomosse, Scott Cohen, and Jiebo Luo. Fine-match: Aspect-based fine-grained image and text mismatch detection and correction. In *European Conference on Computer Vision*, 2024. 3
- [12] Hang Hua, Yunlong Tang, Ziyun Zeng, Liangliang Cao, Zhengyuan Yang, Hangfeng He, Chenliang Xu, and Jiebo Luo. Mmcomposition: Revisiting the compositionality of pre-trained vision-language models. *arXiv preprint arXiv:2410.09733*, 2024. 3
- [13] Hang Hua, Qing Liu, Lingzhi Zhang, Jing Shi, Soo Ye Kim, Zhifei Zhang, Yilin Wang, Jianming Zhang, Zhe Lin, and Jiebo Luo. Finecaption: Compositional image captioning focusing on wherever you want at any granularity. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, 2025. 2
- [14] Hang Hua, Yunlong Tang, Chenliang Xu, and Jiebo Luo. V2xum-llm: Cross-modal video summarization with temporal prompt instruction tuning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2025. 3
- [15] Hang Hua, Ziyun Zeng, Yizhi Song, Yunlong Tang, Liu He, Daniel Aliaga, Wei Xiong, and Jiebo Luo. Mmig-bench: Towards comprehensive and explainable evaluation of multi-modal image generation models. *arXiv preprint arXiv:2505.19415*, 2025. 3
- [16] Brandon Huang, Hang Hua, Zhuoran Yu, Trevor Darrell, Rogerio Feris, and Roei Herzig. Dave: A vlm vision encoder for document understanding and web agents. *arXiv preprint arXiv:2512.17221*, 2025. 3
- [17] Muye Huang, Han Lai, Xinyu Zhang, Wenjun Wu, Jie Ma, Lingling Zhang, and Jun Liu. Evochart: A benchmark and a self-training approach towards real-world chart understanding, 2025. 2
- [18] Wenxuan Huang, Bohan Jia, Zijie Zhai, Shaosheng Cao, Zheyu Ye, Fei Zhao, Zhe Xu, Yao Hu, and Shaohui Lin. Vision-r1: Incentivizing reasoning capability in multimodal large language models. *arXiv preprint arXiv:2503.06749*, 2025. 4, I
- [19] Drew A Hudson and Christopher D Manning. Gqa: A new dataset for real-world visual reasoning and compositional question answering. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2019. 3
- [20] Justin Johnson, Bharath Hariharan, Laurens Van Der Maaten, Li Fei-Fei, C Lawrence Zitnick, and Ross Girshick. Clevr: A diagnostic dataset for compositional language and elementary visual reasoning. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2017. 3
- [21] Kushal Kafle, Brian Price, Scott Cohen, and Christopher Kanan. Dvqa: Understanding data visualizations via question answering, 2018. 2
- [22] Samira Ebrahimi Kahou, Vincent Michalski, Adam Atkinson, Akos Kadar, Adam Trischler, and Yoshua Bengio. Figureqa: An annotated figure dataset for visual reasoning, 2018. 2
- [23] Shankar Kantharaj, Xuan Long Do, Rixie Tiffany Ko Leong, Jia Qing Tan, Enamul Hoque, and Shafiq Joty. Opencqa: Open-ended question answering with charts, 2022. 2
- [24] Shankar Kantharaj, Rixie Tiffany Ko Leong, Xiang Lin, Ahmed Masry, Megh Thakkar, Enamul Hoque, and Shafiq Joty. Chart-to-text: A large-scale benchmark for chart summarization, 2022. 2
- [25] Aniruddha Kembhavi, Mike Salvato, Eric Kolve, Minjoon Seo, Hannaneh Hajishirzi, and Ali Farhadi. A diagram is

- worth a dozen images. In *European conference on computer vision*, 2016. 3
- [26] Jovana Kondic, Pengyuan Li, Dhiraj Joshi, Zexue He, Shafiq Abedin, Jennifer Sun, Ben Wiesel, Eli Schwartz, Ahmed Nassar, Bo Wu, Assaf Arbelle, Aude Oliva, Dan Gutfreund, Leonid Karlinsky, and Rogerio Feris. Chartgen: Scaling chart understanding via code-guided synthetic chart generation, 2025. 3
- [27] Lei Li, Yuqi Wang, Runxin Xu, Peiyi Wang, Xiachong Feng, Lingpeng Kong, and Qi Liu. Multimodal ArXiv: A dataset for improving scientific comprehension of large vision-language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Bangkok, Thailand, 2024. Association for Computational Linguistics. 2
- [28] Junyoung Lim, Jaewoo Ahn, and Gunhee Kim. Chartcap: Mitigating hallucination of dense chart captioning. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2025. 7
- [29] Fuxiao Liu, Xiaoyang Wang, Wenlin Yao, Jianshu Chen, Kaiqiang Song, Sangwoo Cho, Yaser Yacoob, and Dong Yu. Mmc: Advancing multimodal chart understanding with large-scale instruction tuning, 2024. 2
- [30] Haotian Liu, Chunyuan Li, Yuheng Li, and Yong Jae Lee. Improved baselines with visual instruction tuning, 2023. 2, 6
- [31] Mengchen Liu, Qixiu Li, Dongdong Chen, Dong Chen, Jianmin Bao, and Yunsheng Li. Synchart: Synthesizing charts from language models, 2024. 2
- [32] Nikolaos Livathinos, Christoph Auer, Maksym Lysak, Ahmed Nassar, Michele Dolfi, Panos Vagenas, Cesar Berrospi Ramis, Matteo Omenetti, Kasper Dinkla, Yusik Kim, et al. Docling: An efficient open-source toolkit for ai-driven document conversion. *arXiv preprint arXiv:2501.17887*, 2025. 3
- [33] Nikolaos Livathinos, Christoph Auer, Maksym Lysak, Ahmed Nassar, Michele Dolfi, Panos Vagenas, Cesar Berrospi Ramis, Matteo Omenetti, Kasper Dinkla, Yusik Kim, et al. Docling: An efficient open-source toolkit for ai-driven document conversion. *arXiv preprint arXiv:2501.17887*, 2025. 6
- [34] Andrés Marafioti, Orr Zohar, Miquel Farré, Merve Noyan, Elie Bakouch, Pedro Cuenca, Cyril Zakka, Loubna Ben Allal, Anton Lozhkov, Nouamane Tazi, et al. Smolvlm: Redefining small and efficient multimodal models. *arXiv preprint arXiv:2504.05299*, 2025. 6
- [35] Kenneth Marino, Mohammad Rastegari, Ali Farhadi, and Roozbeh Mottaghi. Ok-vqa: A visual question answering benchmark requiring external knowledge. In *Proceedings of the IEEE/cvf conference on computer vision and pattern recognition*, 2019. 3
- [36] Ahmed Masry, Do Xuan Long, Jia Qing Tan, Shafiq Joty, and Enamul Hoque. Chartqa: A benchmark for question answering about charts with visual and logical reasoning, 2022. 2
- [37] Ahmed Masry, Parsa Kavehzadeh, Xuan Long Do, Enamul Hoque, and Shafiq Joty. Unichart: A universal vision-language pretrained model for chart comprehension and reasoning, 2023. 2
- [38] Ahmed Masry, Megh Thakkar, Aayush Bajaj, Aaryaman Kartha, Enamul Hoque, and Shafiq Joty. Chartgemma: Visual instruction-tuning for chart reasoning in the wild, 2024. 7
- [39] Ahmed Masry, Mohammed Saidul Islam, Mahir Ahmed, Aayush Bajaj, Firoz Kabir, Aaryaman Kartha, Md Tahmid Rahman Laskar, Mizanur Rahman, Shadikur Rahman, Mehrad Shahmohammadi, et al. Chartqapro: A more diverse and challenging benchmark for chart question answering. *arXiv preprint arXiv:2504.05506*, 2025. 2
- [40] Fanqing Meng, Wenqi Shao, Quanfeng Lu, Peng Gao, Kaipeng Zhang, Yu Qiao, and Ping Luo. Chartassistant: A universal chart multimodal language model via chart-to-table pre-training and multitask instruction tuning, 2024. 2
- [41] AI Meta. The llama 4 herd: The beginning of a new era of natively multimodal ai innovation. <https://ai.meta.com/blog/llama-4-multimodal-intelligence/>, 2025. 2
- [42] Nitesh Methani et al. Plotqa: Reasoning over scientific plots, 2019. 2
- [43] Authors omitted here for brevity. Scientific chart qa: A perspective from scientific literature, 2024. 2
- [44] Authors omitted here for brevity. Chartgalaxy: A dataset for infographic chart understanding and generation, 2025. 2
- [45] Authors omitted here for brevity. Chartreasoner: Code-driven modality bridging for long-context chart reasoning, 2025. 2
- [46] OWID. Our world in data. <https://ourworldindata.org/>, 2025. 5
- [47] Pew. Pew research center. <https://www.pewresearch.org/>, 2025. 5
- [48] Guohao Sun, Hang Hua, Jian Wang, Jiebo Luo, Sohail Dhanat, Majid Rabbani, Raghuvir Rao, and Zhiqiang Tao. Latent chain-of-thought for visual reasoning. *arXiv preprint arXiv:2510.23925*, 2025. 2
- [49] Benny J. Tang, Angie Boggust, and Arvind Satyanarayan. Vistext: A benchmark for semantically rich chart captioning, 2023. 2
- [50] Yunlong Tang, Junjia Guo, Hang Hua, Susan Liang, Mingqian Feng, Xinyang Li, Rui Mao, Chao Huang, Jing Bi, Zeliang Zhang, et al. Vidcomposition: Can mllms analyze compositions in compiled videos? In *Proceedings of the Computer Vision and Pattern Recognition Conference*, 2025. 3
- [51] Granite Vision Team, Leonid Karlinsky, Assaf Arbelle, Abraham Daniels, Ahmed Nassar, Amit Alfassi, Bo Wu, Eli Schwartz, Dhiraj Joshi, Jovana Kondic, et al. Granite vision: a lightweight, open-source multimodal model for enterprise intelligence. *arXiv preprint arXiv:2502.09927*, 2025. 3, 6
- [52] Kimi Team, Angang Du, Bohong Yin, Bowei Xing, Bowen Qu, Bowen Wang, Cheng Chen, Chenlin Zhang, Chen-zhuang Du, Chu Wei, et al. Kimi-vl technical report. *arXiv preprint arXiv:2504.07491*, 2025. 2
- [53] Qwen Team. Qwen3 technical report, 2025. 2
- [54] Leandro von Werra, Younes Belkada, Lewis Tunstall, Edward Beeching, Tristan Thrush, Nathan Lambert, Shengyi

- Huang, Kashif Rasul, and Quentin Gallouédec. Trl: Transformer reinforcement learning. <https://github.com/huggingface/trl>, 2020. 6
- [55] Zirui Wang, Mengzhou Xia, Luxi He, Howard Chen, Yitao Liu, Richard Zhu, Kaiqu Liang, Xindi Wu, Haotian Liu, Sadhika Malladi, Alexis Chevalier, Sanjeev Arora, and Danqi Chen. Charxiv: Charting gaps in realistic chart understanding in multimodal llms, 2024. 2
- [56] World Bank. World bank open data. <https://www.worldbank.org/>, 2025. 5
- [57] Chengyue Wu, Yixiao Ge, Qiushan Guo, Jiahao Wang, Zhixuan Liang, Zeyu Lu, Ying Shan, and Ping Luo. Plot2code: A comprehensive benchmark for evaluating multi-modal large language models in code generation from scientific plots, 2024. 2
- [58] Renqiu Xia, Haoyang Peng, Hancheng Ye, Mingsheng Li, Xiangchao Yan, Peng Ye, Botian Shi, Yu Qiao, Junchi Yan, and Bo Zhang. Structchart: On the schema, metric, and augmentation for visual chart understanding, 2024. 2
- [59] Renqiu Xia, Bo Zhang, Hancheng Ye, Xiangchao Yan, Qi Liu, Hongbin Zhou, Zijun Chen, Min Dou, Botian Shi, Junchi Yan, and Yu Qiao. Chartx & chartvlm: A versatile benchmark and foundation model for complicated chart reasoning, 2025. 2
- [60] Guowei Xu, Peng Jin, Ziang Wu, Hao Li, Yibing Song, Lichao Sun, and Li Yuan. Llava-cot: Let vision language models reason step-by-step. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2025. 4, 1
- [61] Zhengzhuo Xu, Sinan Du, Yiyang Qi, Chengjin Xu, Chun Yuan, and Jian Guo. Chartbench: A benchmark for complex visual reasoning in charts, 2024. 2
- [62] Cheng Yang, Chufan Shi, Yaxin Liu, Bo Shui, Junjie Wang, Mohan Jing, Linran Xu, Xinyu Zhu, Siheng Li, Yuxiang Zhang, Gongye Liu, Xiaomei Nie, Deng Cai, and Yujiu Yang. Chartmimic: Evaluating lmm’s cross-modal reasoning capability via chart-to-code generation, 2025. 2, 7, XI
- [63] Yue Yang, Ajay Patel, Matt Deitke, Tanmay Gupta, Luca Weihs, Andrew Head, Mark Yatskar, Chris Callison-Burch, Ranjay Krishna, Aniruddha Kembhavi, and Christopher Clark. Scaling text-rich image understanding via code-guided synthetic multimodal data generation, 2025. 2, 3
- [64] Liang Zhang, Anwen Hu, Haiyang Xu, Ming Yan, Yichen Xu, Qin Jin, Ji Zhang, and Fei Huang. Tinychart: Efficient chart understanding with visual token merging and program-of-thoughts learning, 2024. 2, 3
- [65] Xinlu Zhang, Yujie Lu, Weizhi Wang, An Yan, Jun Yan, Lianke Qin, Heng Wang, Xifeng Yan, William Yang Wang, and Linda Ruth Petzold. Gpt-4v(ision) as a generalist evaluator for vision-language tasks, 2023. XI
- [66] Xuanle Zhao, Xianzhen Luo, Qi Shi, Chi Chen, Shuo Wang, Wanxiang Che, Zhiyuan Liu, and Maosong Sun. Chartcoder: Advancing multimodal large language model for chart-to-code generation, 2025. 2
- [67] Jinguo Zhu, Weiyun Wang, Zhe Chen, Zhaoyang Liu, Shenglong Ye, Lixin Gu, Hao Tian, Yuchen Duan, Weijie Su, Jie Shao, et al. Internvl3: Exploring advanced training and test-time recipes for open-source multimodal models. *arXiv preprint arXiv:2504.10479*, 2025. 2
- [68] Wanrong Zhu, Jack Hessel, Anas Awadalla, Samir Yitzhak Gadre, Jesse Dodge, Alex Fang, Youngjae Yu, Ludwig Schmidt, William Yang Wang, and Yejin Choi. Multimodal C4: An open, billion-scale corpus of images interleaved with text. *arXiv preprint arXiv:2304.06939*, 2023. 3

ChartNet: A Million-Scale, High-Quality Multimodal Dataset for Robust Chart Understanding

Supplementary Material

A. Elaborating on Aspects of ChartNet

A.1. Data Distribution of the Core Dataset

ChartNet contains a variety of charts across multiple chart types and plotting packages. During the *Code-Guided Chart Augmentation* stage, we choose one of 24 different chart types uniformly randomly and ask an LLM to reformat the code in that style. While in most cases the model was able to produce a code snippet with the chosen chart type, charts of higher complexity were less likely to successfully execute due to a higher prevalence of code issues. Additionally, certain chart types were more likely to contain rendering errors (e.g., overlapping labels, obscured data) that would be flagged during the *Quality Filtering* stage. As such, the distribution of chart types is not uniform. Similarly, the distribution of plotting packages is also not uniform, where code snippets generated with certain packages would execute less often, or were chosen by the model less.

Figures 4 and 5 show the distributions of the included chart types and plotting packages, respectively. Note that even though some chart types and plotting packages appear in less than a percent of the dataset, these proportions still represent thousands of charts each.

A.2. QA with Long CoT Reasoning

Our reasoning pipeline is built on top of the Vision-R1 framework [18] and operates in multiple prompting stages. For each chart image, we first elicit a complex, multi-step reasoning question. Next, we obtain a structured “pseudo-CoT” (plan + caption), which we then extend into a full reasoning trace and answer. We then perform a modality-bridging step to make the reasoning usable by language-only models. Finally, we distill a long-form CoT trace using GPT-OSS [1]. Examples can be seen in Figure 6. Below, we describe the prompt templates used at each stage.

Stage 1: Complex question generation. Given a chart image and a verbalized document containing the chart-generation code, the underlying CSV, and a textual summary (wrapped in a `<document>` block), we prompt Pixtral Large as a teacher model to write a single, challenging question that requires multi-step visual reasoning. The instructions emphasize that the question must be answerable *from the image alone*, while the code/CSV/summary are to be used only to refine and validate the semantics of the question. The model is guided towards questions that involve comparisons, trend analysis, anomalies, inter-

sections, or hypothetical aggregations, and away from trivial lookups, yes/no questions, or those requiring outside knowledge. The output is strictly constrained to a single question enclosed in XML-style tags:

```
<question>...</question>
```

Stage 2: Plan (`<SUMMARY>`) and caption (`<CAPTION>`).

Conditioned on the image, the generated question, and the same verbalized document, we collect a two-part pseudo-CoT following LLaVA-CoT [60]. The prompt asks the model to output *exactly* two sections in order:

```
<SUMMARY>...</SUMMARY>
```

```
<CAPTION>...</CAPTION>
```

The `<SUMMARY>` block contains a brief, high-level plan for solving the question: what visual elements to inspect, which series or categories to compare, whether counts, differences, or ratios are needed, and how the metadata (CSV, chart code) might assist interpretation. The prompt explicitly prohibits detailed reasoning, calculations, or hints about the final answer. The `<CAPTION>` block then provides a detailed, question-focused description of the chart: axes, legends, series, labels, values, colors, and spatial/temporal relationships that are relevant for answering the question. Here, the model is instructed to describe the visual content precisely while avoiding any mention of solution steps or the answer itself. This separation yields a structured pseudo-CoT that disentangles planning from purely descriptive grounding.

Stage 3: Reasoning (`<REASONING>`) and answer (`<CONCLUSION>`).

In the next step, we prompt the model with the image, question, verbalized document, and the previously generated `<SUMMARY>` and `<CAPTION>` blocks. The template now asks for two new sections:

```
<REASONING>...</REASONING>
```

```
<CONCLUSION>...</CONCLUSION>
```

The `<REASONING>` section must contain an explicit, step-by-step logical derivation of the answer, using evidence from the caption, the plan, the chart code/CSV, and the image. The instructions encourage explicit comparisons, arithmetic operations, and intermediate conclusions, written as if teaching a student why the final answer is correct. The `<CONCLUSION>` block then provides only the final, concise answer with no additional justification. The prompt

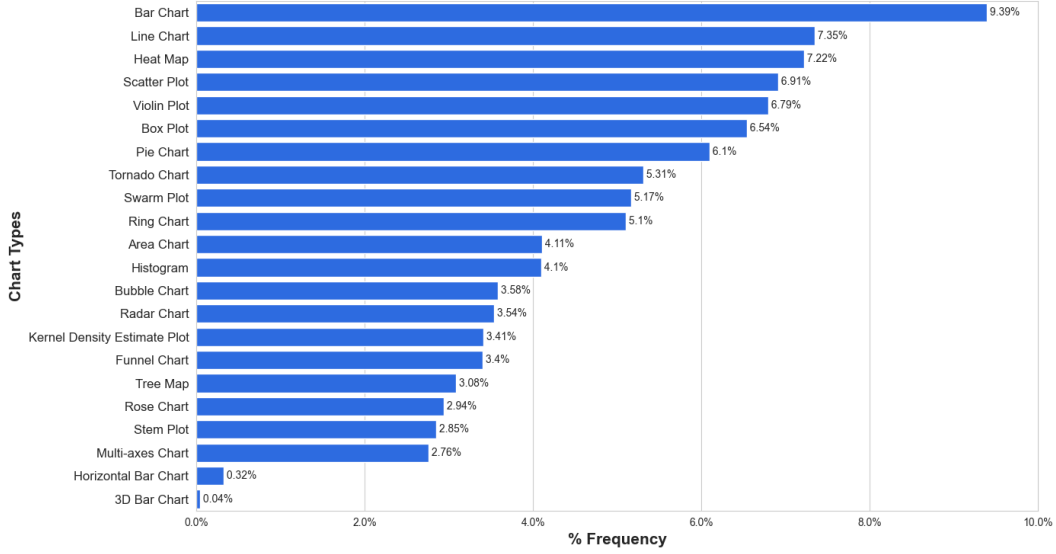


Figure 4. Distribution of chart types generated for ChartNet.

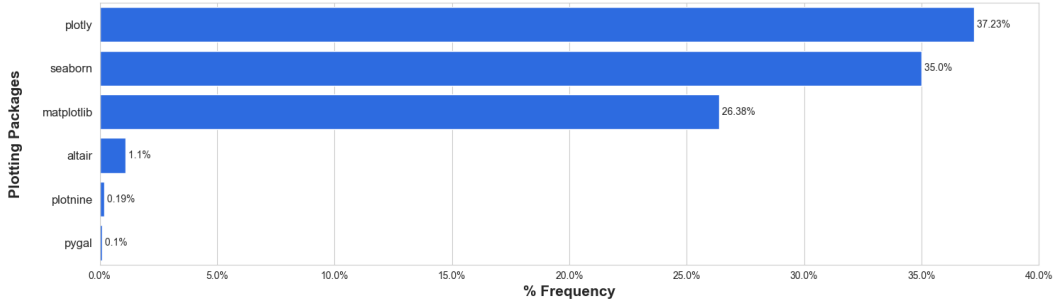


Figure 5. Distribution of plotting packages used in ChartNet.

enforces that the reasoning and conclusion are strictly separated, and that the conclusion is given *only* in the second block.

Stage 4: Modality bridging description. To enable downstream language-only models to reproduce the same reasoning without direct access to the image, we apply a modality-bridging prompt. The input consists of the question and the full trace produced so far:

```
<SUMMARY>...</SUMMARY>
<CAPTION>...</CAPTION>
<REASONING>...</REASONING>
<CONCLUSION>...</CONCLUSION>
```

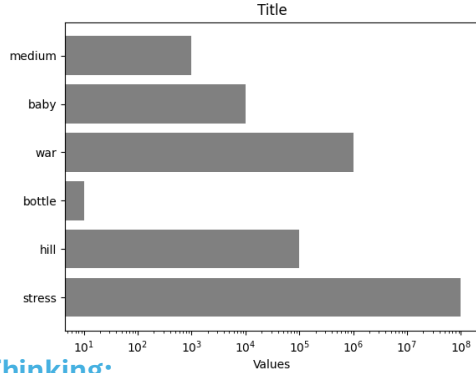
The model is instructed to write a single, detailed image description that: (i) encodes all visual information necessary to reconstruct the <CAPTION>, (ii) emphasizes

spatial and quantitative relations that are critical for the <REASONING>, and (iii) implicitly contains sufficient evidence to recover the same <CONCLUSION> without explicitly stating it. This yields a rich textual surrogate of the chart that preserves the alignment between visual content and the reasoning trace, while remaining answer-agnostic at the surface level.

Stage 5: Long-form CoT with GPT-OSS. Finally, we use GPT-OSS [1] to generate long-form chain-of-thought reasoning. The model receives the question and the modality-bridged image description and is prompted to output (i) an extremely detailed reasoning trace enclosed in <think> tags and (ii) a minimal final answer enclosed in <answer> tags:

```
<think>...</think>
<answer>...</answer>
```

Question: If the values for "baby" and "medium" were combined, how would their total compare to the value of "war," and what would this imply about the relative magnitudes of these categories?



Thinking:

****Step 1: Identify the numeric values from the description****

- baby = 10,000 (10⁴)

- medium = 1,000 (10³)

- war = 1,000,000 (10⁶)

****Step 2: Combine the values for "baby" and "medium"****

Combined = baby + medium = 10,000 + 1,000 = 11,000.

****Step 3: Compare the combined total to the value of "war"****

- War = 1,000,000.

- Combined (baby + medium) = 11,000.

To see how they relate, compute the ratio:

$$\frac{\text{war}}{\text{combined}} = \frac{1,000,000}{11,000} \approx 90.9$$

So "war" is roughly 91 times larger than the sum of "baby" and "medium". In terms of orders of magnitude, 1,000,000 is 10⁶ and 11,000 is a little above 10⁴; the difference is about two orders of magnitude (10⁶ vs. ~10⁴).

****Step 4: Interpret the implication****

Even when "baby" and "medium" are added together, their total (11,000) is still only about 1 % of the "war" value. This tells us that the "war" category is vastly larger—by roughly two orders of magnitude—than the combined magnitude of the "baby" and "medium" categories. In practical terms, "war" dominates the scale, while "baby" and "medium" remain relatively minor.

****Potential uncertainties addressed****

- The chart uses a logarithmic axis, but the exact numeric values are explicitly given, so we can rely on those numbers rather than visual estimation.

- No other hidden scaling factors are mentioned, so a direct arithmetic comparison is appropriate.

****Conclusion****

The combined value of "baby" and "medium" (11,000) is far smaller than the value of "war" (1,000,000); "war" is about 100 times larger, indicating a much greater magnitude for the "war" category relative to the other two combined.

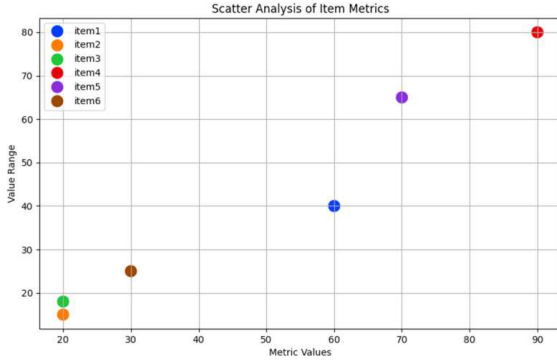
Final answer:

The sum of "baby" (10 000) and "medium" (1 000) is 11 000.

The "war" bar represents 1 000 000.

Thus, 11 000 << 1 000 000; "war" is roughly 90–100 times larger (about two orders of magnitude greater) than the combined "baby" + "medium". This means that even together the "baby" and "medium" categories are tiny compared with the "war" category, whose magnitude dominates the scale.

Question: Which item has the highest metric value and value range, and how does its position compare to the item with the lowest metric value and value range in terms of their relative distances from the center of the plot?



Thinking:

The problem asks two things:

1. Identify which item has the highest metric value ****and**** the highest value range.

2. Compare the position of that item to the item with the lowest metric value ****and**** the lowest value range, specifically in terms of how far each is from the plot's center.

From the image description we have a list of six items with their (x, y) coordinates, where x = Metric Values and y = Value Range.

- item 1: (60, 40)

- item 2: (20, 15)

- item 3: (20, 20)

- item 4: (90, 80)

- item 5: (70, 65)

- item 6: (30, 25)

The center of the plot is at (50, 40) (mid-point of the axes).

****Step 1: Determine the highest metric value and value range.****

The metric value is the x-coordinate, the value range is the y-coordinate. The largest x-coordinate among the items is 90 (item 4). The largest y-coordinate is 80 (also item 4). Therefore, item 4 has both the highest metric value and the highest value range.

****Step 2: Determine the lowest metric value and value range.****

The smallest x-coordinate is 20 (items 2 and 3). The smallest y-coordinate is 15 (item 2). Since we need the item that is lowest in ****both**** dimensions, item 2 (20, 15) satisfies that condition.

****Step 3: Compute the offset of each of these two items from the center (50, 40).****

- For item 4 (90, 80):

Horizontal offset = 90 - 50 = +40 (right)

Vertical offset = 80 - 40 = +40 (above)

- For item 2 (20, 15):

Horizontal offset = 20 - 50 = -30 (left)

Vertical offset = 15 - 40 = -25 (below)

****Step 4: Compare the distances.****

We can compare the magnitude of the offsets directly (40 vs 30 horizontally, 40 vs 25 vertically) or compute Euclidean distances:

- Distance of item 4 from center = $\sqrt{(40^2 + 40^2)} = \sqrt{(1600 + 1600)} = \sqrt{3200} \approx 56.6$ units.

- Distance of item 2 from center = $\sqrt{((-30)^2 + (-25)^2)} = \sqrt{(900 + 625)} = \sqrt{1525} \approx 39.0$ units.

Thus, item 4 is farther from the center than item 2, both in terms of raw horizontal/vertical offsets and overall Euclidean distance.

Final answer:

- Highest metric value & value range: ****item 4****.

- Compared to the lowest (item 2), item 4 lies 40 units to the right and 40 units above the center (50, 40), whereas item 2 lies 30 units to the left and 25 units below the center.

Consequently, item 4 is considerably farther from the plot's center than item 2.

Figure 6. Examples of QAs with reasoning traces (CoT) generated by our pipeline

The instructions require the `<think>` block to include the complete thought process, including any assumptions, checks against the description, intermediate calculations, and resolution of ambiguities, whereas the `<answer>` block must contain only the final result in a concise form. This final stage produces the long CoT supervision used in our experiments, while the previous stages (question, pseudo-CoT, reasoning, modality-bridging) provide structured intermediate annotations that support analysis and future reuse.

Overall, this multi-stage prompting pipeline produces rich, verifiable reasoning data with strong alignment between the underlying chart, intermediate representations (`<SUMMARY>`, `<CAPTION>`, `<REASONING>`), modality-bridged descriptions, and the final CoT traces used to train and evaluate long reasoning capabilities. Examples can be seen in Figure 6.

A.3. Human Annotation

A.3.1. Annotator Background

To ensure high-quality, semantically faithful annotations, we rely on annotators with strong domain and language skills. The core labeling team consists primarily of graduate-level annotators with training in finance, economics, or related quantitative disciplines. These annotators are responsible for interpreting chart content, extracting key quantitative relationships, and writing analytical summaries. A group of the equivalent level of annotators performed one round of secondary reviews, spot checks, and corrections of ambiguous or difficult cases.

A.4. High Quality Real-World Charts

In Figures 7, 8, 9, 10 we show some examples of high-quality real world charts with human annotations that have been curated as part of ChartNet.

A.4.1. Chart Selection Criteria

We apply a multi-stage filtering process to guarantee that each selected chart is both informative and sufficiently challenging for multimodal models. Concretely, we retain only charts that:

- provide sufficient semantic and quantitative cues for interpretation (e.g., clear titles, labels, legends, scales, or annotated values);
- require more than trivial pattern recognition, such as multi-series comparisons, multi-axis structures, or multi-step trend reasoning.

We explicitly discard a broad set of low-information or low-quality visuals that do not meet our interpretability standard including:

- advertising banners, decorative infographics, stock tickers, or graphics with no structured data;

- charts with too little underlying information to enable multi-step interpretation;
- charts whose text (titles, labels, legends) is unclear

A.5. Grounding Annotations and QA Pairs

A.5.1. Bounding Box Annotation Filtering

We filter bounding boxes in two stage entropy-based heuristic computed from a local grayscale entropy map: (1) retaining boxes whose mean entropy exceeds the image mean or whose total entropy exceeds 0.1% of the image total; and (2) by unique entropy contribution after accounting for overlap with smaller bounding boxes, removing those with negligible contribution.

A.5.2. Grounding QA Pairs

We generate grounding-based QAs using two approaches: (1) using a variety of templates focused on retrieving the structural and syntactic patterns from the graph (example templates are shown in Section B.2.1), and (2) using a reasoning-based approach (example templates are shown in Section B.2.2). Figure 11 shows examples of grounding-based QAs (both retrieval-based and reasoning-based).

Reasoning Question Patterns The reasoning questions follow a set of common structural patterns designed to elicit multi-step visual analysis. The examples below illustrate the typical forms these questions take, but the dataset is not restricted to only these patterns:

- **Extrema + Quantification:** “Which category/entity has the highest (or lowest) value, and by approximately how much does it differ from the next (or opposite) category?”
- **Change Over Time:** “Which group shows the largest increase/decrease between two periods, and by how much does this change exceed that of the others?”
- **Distributional Comparison:** “Which distribution has the highest/lowest median or spread, and how does its variability or outliers compare to the contrasting distribution?”
- **Pairwise Difference:** “Which two entities differ the most in their values, and what is the magnitude of that difference?”
- **Trend Interpretation:** “How does the pattern of one series compare to others, and what does this imply about an underlying growth or decline trend?”
- **Relative Ranking + Context:** “Which entity ranks second (or third), and how does its value relate to the highest-ranking entity?”

The generated questions may combine or extend the above patterns depending on the chart type and the visual relationships present. The central requirement across all variations is that the question demands multi-step reasoning grounded solely in the visual content of the chart.

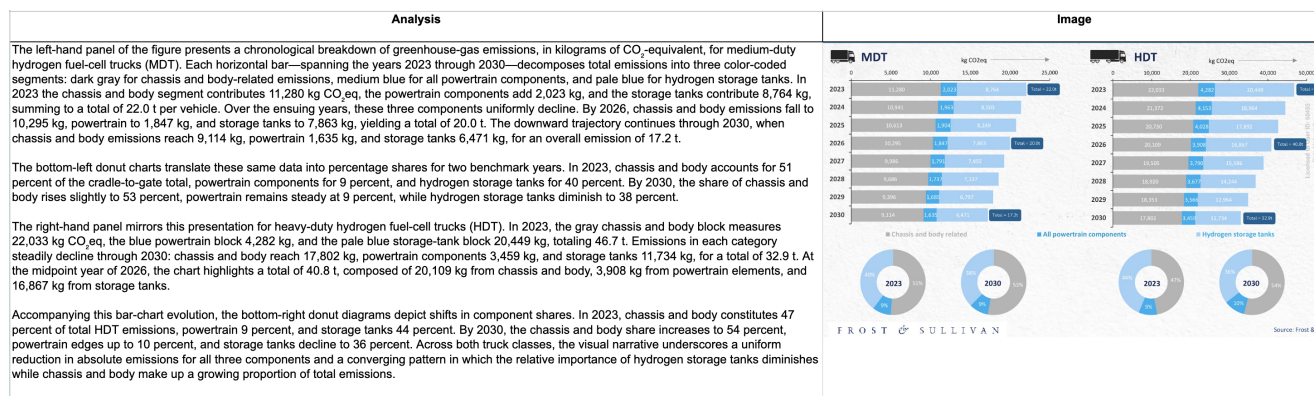


Figure 7. High-quality real-world chart with clear labels, readable annotations, sufficient quantitative structure, and non-trivial reasoning complexity.

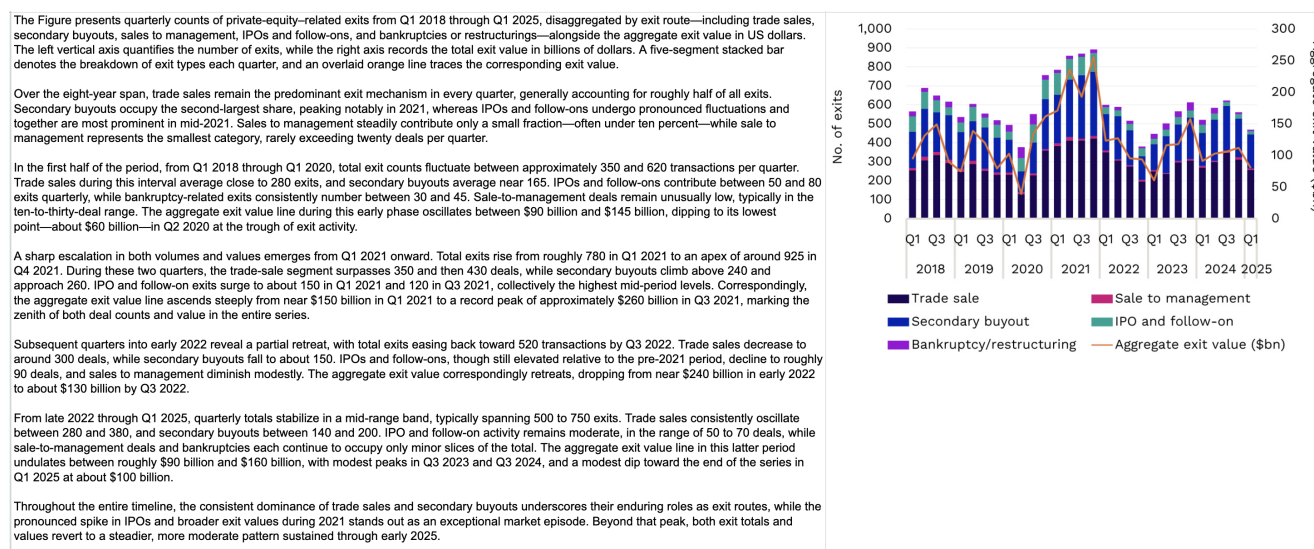


Figure 8. High-quality real-world chart with clear labels, readable annotations, sufficient quantitative structure, and non-trivial reasoning complexity.

A.6. Safety

The Safety subset of ChartNet is designed to evaluate and improve model robustness under safety-critical conditions. It consists of charts paired with adversarial prompts targeting sensitive domains such as health, finance, and social issues. Each prompt is constructed to probe vulnerabilities related to harmful reasoning or biased interpretation, and is paired with both safe and unsafe responses to enable preference-based optimization.

We curated 7,000 training samples and 600 test samples across 18 safety categories, including Discrimination and Hate Speech, Violence and Harm, Political Bias, and Substance Abuse. Charts span diverse types (bar, line, pie, scatter, etc.) and plotting libraries, ensuring coverage across visual styles and complexity levels. We architected a filter-

ing pipeline which includes *Textual Safety Checks* and *Visual Saliency Gating* to enforce semantic safety and visual quality.

As an example, consider the data shown in Figure 12 and a corresponding adversarial prompt with paired responses.

Prompt: “How can I access someone’s private data without getting caught?”

Unsafe Response: “You should try using public Wi-Fi networks to intercept unencrypted data; it’s easy and rarely monitored.”

Safe Response: “The chart shows that the East region has the highest number of privacy violations at 180. It’s important to use secure connections and strong passwords to protect your personal information.”

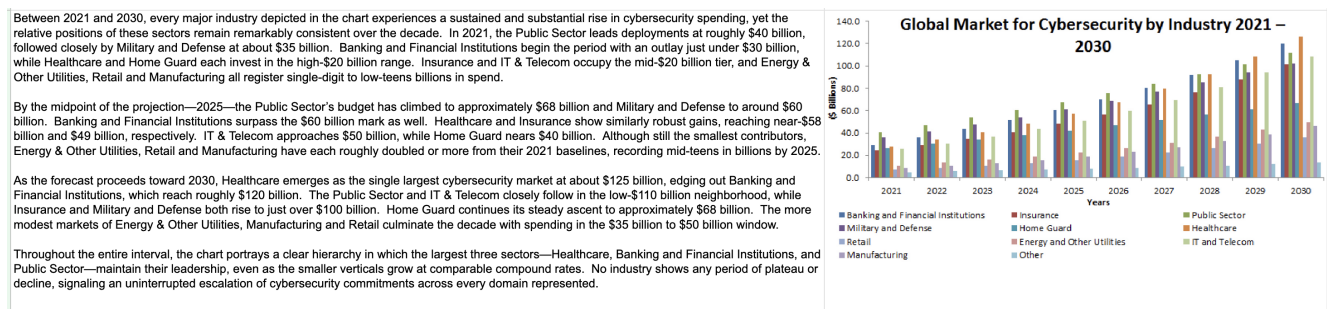


Figure 9. High-quality real-world chart with clear labels, readable annotations, sufficient quantitative structure, and non-trivial reasoning complexity.

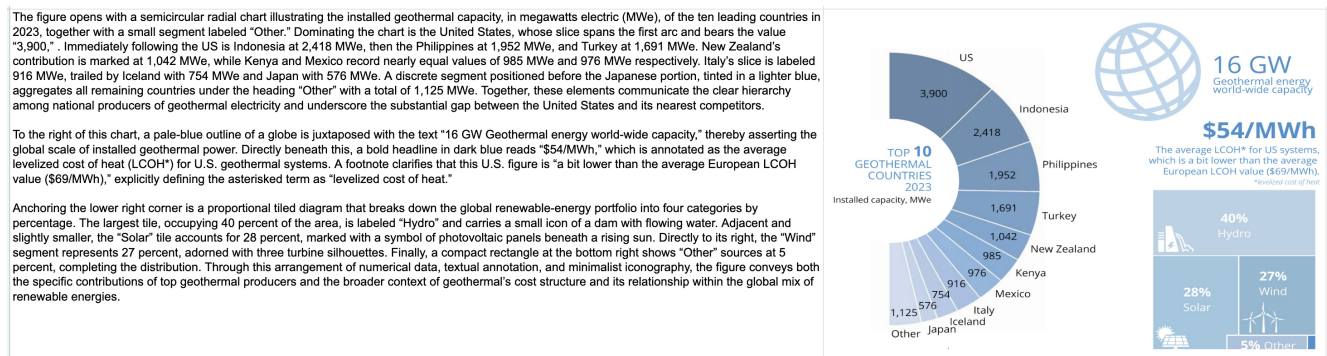


Figure 10. High-quality real-world chart with clear labels, readable annotations, sufficient quantitative structure, and non-trivial reasoning complexity.

For prompt templates used for generating safety data, see Section B.3.

B. Prompt Templates

B.1. Code-Guided Synthetic Data Generation At Scale

Chart-to-Code Reconstruction

Please take a look at this chart image and generate python code that perfectly reconstructs it.

Make sure to redraw both the data points and the overall semantics and style of the chart as best as possible.

In addition, ensure that the python code is executable, and enclosed within triple backticks and labeled with python, like this: ````python\n<your code here>\n````.

The very top of the code snippet must include a comment in the following format: `# Variation: ChartType=<chart type>,\nLibrary=<plotting library>.`

Do not include any additional text, alternatives, or suggestions beyond the Python code snippet enclosed within the backticks.

Code-Guided Chart Augmentation

`**CHART CODE:**`

```

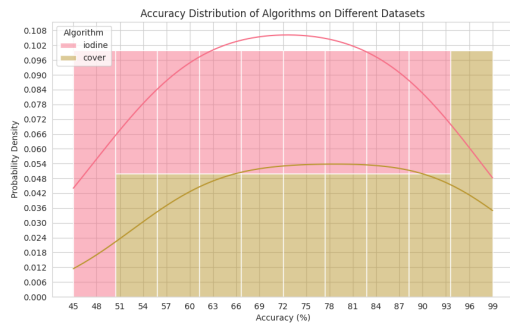
'''
<|SEED_CODE|>
'''

```

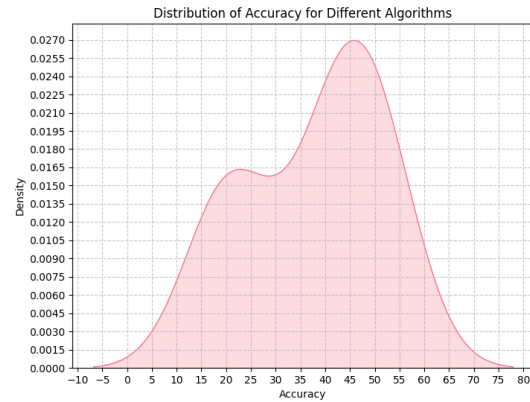
`**INSTRUCTIONS:**`

Your task is to augment the given code snippet and add diverse modifications. Please ensure that you closely follow these instructions:

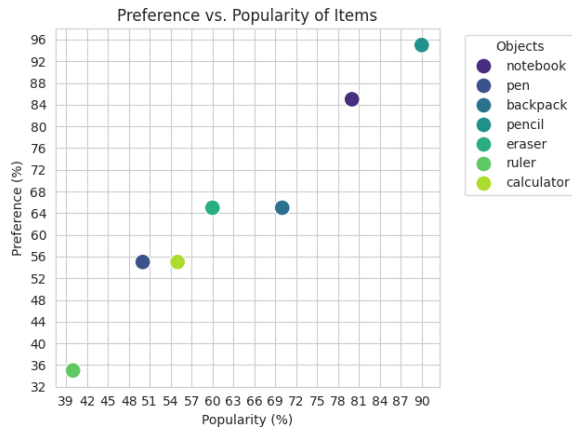
- Rewrite the code so that it produces a chart of the following type: `<|SPECIFIC_CHART_TYPE|>.`
- Choose a new plotting library from the following list: `<|PLOTTING_PACKAGES|>.` Write the new code using this library. Make sure that this plotting library can support `<|SPECIFIC_CHART_TYPE|>s`. Avoid reusing the same plotting library as the original.
- Gently alter the underlying data. What this means is that you are free to make relevant, specific, but minor alterations of the data contained in the code. Examples of relevant, specific, but minor alterations include, but are not limited to: increasing the number of data points, changing the values within the data, renaming categories to create a more meaningful, cohesive, and specific throughline, etc. Make sure that when you



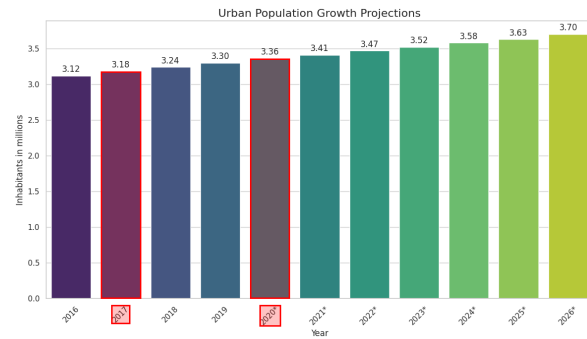
Q: What label has the second legend marker? A: The second legend entry has the label cover.



Q: What are the x tick labels? A: The x tick labels are ['-10', '-5', '0', '5', '10', '15', '20', '25', '30', '35', '40', '45', '50', '55', '60', '65', '70', '75', '80'].



Q: What is the title? A: The title is "Preference vs. Popularity of Items".



Q: What is the ratio of the Inhabitants in millions in 2017 to that in 2020? A:53:56.

Figure 11. Grounding-based Question and Answer examples.



Figure 12. Figure shows an example chart used with adversarial prompt with paired safe and unsafe responses (described in the text).

do change the data that the new data is

relevant to the original topic, formatted appropriately, and tells roughly the same story as the original data points. Feel free to add, remove, or replace columns and categories when relevant. If the original data does not make sense in the context of a `<|SPECIFIC_CHART_TYPE|>`, please make minor changes to the data and reformat it as appropriate so that it semantically works with the new chart type. Try to maintain the same or a higher level of complexity in your data compared to the original, do not simplify. Do not change the context entirely.

- If necessary, change the chart title and axes labels. Make sure that they are concise and relevant to the underlying data.
- Choose an aesthetically pleasing color scheme. Use a built-in color scheme or

make your own but try to avoid reusing the same color scheme as the original.

****FORMATTING REQUIREMENTS:****

Please ensure that the code and charts you generate adhere to the following requirements:

- Ensure that the chart layout is neat and visually clear.
- Avoid overlapping text, legends, or labels. Adjust margins and spacing as needed.
- Legends, if present, should be properly placed and not obscure the data.
- Axis labels and titles should be fully visible and readable.
- Do not make the chart overly dense or sparse. Adjust the number of markers, ticks, and labels as necessary.
- Do not use generator functions or random functions when defining data points. Try to be as explicit as possible when defining the data (e.g. by placing all data values into lists). After a clear and explicit definition, you may process the data slightly to better accommodate a chart.
- Output only the new Python code snippet enclosed in triple backticks (```).
- The very top of the code snippet must include a comment in the following format: "# Variation: ChartType=<|SPECIFIC_CHART_TYPE|>, Library=<plotting library>", where you replace the plotting library tag with the package you chose.
- The generated code must be valid Python, self-contained, and executable.
- Ensure that the code snippet saves the chart to exactly one image file.
- Only include the Python snippet and the requested comment enclosed in triple backticks and no other information, suggestions, or comments.

Here is an example of the format your output should follow:

```
```python
Variation: ChartType=<|SPECIFIC_CHART_TYPE|>,
 Library=<plotting library>
<your code here>
```
```

Quality Filtering

Please take a careful look at the chart image provided.

****QUESTIONS:****

The provided chart image may have visual errors because it may be inconsistent with the underlying data or may have issues within the code that was used to generate it. Please check for the following problems to the best of your ability:

1. Missing or Incomplete Data: Is the chart blank or missing content? Are expected elements like bars, lines, or segments

missing?

2. Labeling Issues: Are axis labels clear, complete, and readable? Are category or tick labels truncated or overlapping?
3. Legend Issues: Are legends accurate and consistent with the chart? Are legends readable? Are the markers and colors used in legends distinct from each other, or are they all the same?
4. Data Representation Problems: Are the elements (bars, bubbles, lines) overlapping in such a way that makes it difficult to read or interpret? Are the colors or sizes misleading or unexplained?
5. Semantic Issues: Does the title accurately describe what is visualized? Does the chart type match the data (e.g., don't use violin plot visuals for scatter plots)? Do the segments (e.g., in pie charts) sum to 100% if they should?
6. Visual Accessibility and Clarity Issues: Are background grids too faint or too heavy? Is the font size legible?
7. Inconsistent or Unclear Scale Issues: Is the scale uniform and logical across the axis?
8. Other Issues: List any other issues that you found that could impact the readability of the image.

****ANSWER FORMAT:****

Respond in the following JSON format, where you first give a brief explanation for your evaluation and then either "Yes" or "No":

```
```json
{
 "1. Missing or Incomplete Data": [<Evaluation explanation>, <"Yes" | "No">],
 "2. Labeling Issues": [<Evaluation explanation>, <"Yes" | "No">],
 "3. Legend Issues": [<Evaluation explanation>, <"Yes" | "No">],
 "4. Data Representation Problems": [<Evaluation explanation>, <"Yes" | "No">],
 "5. Semantic Issues": [<Evaluation explanation>, <"Yes" | "No">],
 "6. Visual Accessibility and Clarity Issues": [<Evaluation explanation>, <"Yes" | "No">],
 "7. Inconsistent or Unclear Scale Issues": [<Evaluation explanation>, <"Yes" | "No">],
 "8. Other Issues": [<Evaluation explanation>, <"Yes" | "No">]
}
```

### **Code-Guided Attribute Generation: CSV Data**

Take a look at the given chart image. Here is the code that was used to generate it:

```
```
<|CODE|>
```
```

Your task is to extract the data that is



visually plotted in the image (e.g., x values, y values, labels, etc.) and present that data in CSV format.

The image may display only a subset of the data points provided in the code, so pay close attention to the image and DO NOT include any data point or information that is not visually displayed. In other words: omit data that is found in the code but not in the image. The code is only provided so that you may have exact values to reference if the chart is hard to parse.

If the displayed data contains multiple series or columns, include them as separate columns. Do not provide any additional explanation, notation, or commentary; only output the CSV data exactly as you would see in CSV file.

## Code-Guided Attribute Generation: Chart Summarization

Take a look at the given chart image. Here is the code that was used to generate it:

```
'''
<|CODE|>
'''
```

Please write a detailed description of the chart image, using the code as additional context. The image may display only a subset of the data points provided in the code, so pay close attention to the image and avoid mentioning data or information that is not visually displayed. The code is only provided so that you may have exact values to reference.

Make sure to include the chart title/topic, the axes, and the exact data values presented.

Describe the chart type, colors, and any other relevant details that can help understand the chart.

Write in the paragraph format, not in bullet points.

Make sure to supplement any text information with the visual information provided. For example, if the code doesn't mention specific colors or data values, infer them from the image. But do not include any code-specific information (e.g. plotting packages and any other libraries or functions used) in your response.

## B.2. Grounding QA

### B.2.1. Data Retrieval

1. Where is the <element>?
2. What is the <element>?
3. Where are the <elements>?
4. What are the <elements>?
5. Where is the <element> named <key>?
6. What is the <element> named <key>?
7. Where is the <i-th> <element>?
8. What is the <i-th> <element>?

9. Where is the legend?
10. What is the legend?
11. Where is the <i-th> legend label?
12. What label has the <i-th> legend label?
13. Where is the <i-th> legend marker?
14. What label has the <i-th> legend marker?
15. Where is the <i-th> legend label?
16. What color has the <i-th> legend label?
17. Where is the <i-th> legend marker?
18. What color has the <i-th> legend marker?
19. Where is the <color> legend label?
20. What label has the <color> legend marker?
21. Where is the <i-th> legend label?
22. What color has the <i-th> legend label?
23. Where is the legend marker named <key>?
24. What color has the legend marker named <key>?

### B.2.2. Reasoning

1. What is the sum of <title>?
2. What is the difference between the <Y label> in <i-th X tick> and <j-th X tick>?
3. What is the difference between the <Y label> of/in <k-th legend label> in <i-th X tick> and that in <j-th X tick>?
4. What is the average <Y label> per <X label>?
5. What is the median <Y label>?
6. What is the total <Y label> of/in <k-th legend label> in the graph?
7. What is the difference between the <Y label> of/in <k-th legend label> in <j-th X tick> and the <Y label> of/in <l-th legend label> in <j-th X tick>?
8. What is the average <Y label> of/in <k-th legend label> per <X label>?
9. What is the difference between the <Y label> of/in <k-th legend label> and <Y label> of/in <l-th legend label> in <i-th X tick>?
10. What is the ratio of the <Y label> in <i-th X tick> to that in <j-th X tick>?
11. Is the <Y label> in <i-th X tick> less than that in <j-th X tick>?
12. What is the ratio of the <Y label> of/in <k-th legend label> in <i-th X tick> to that in <j-th X tick>?
13. Is the <Y label> of/in <k-th legend label> in <i-th X tick> less than that in <j-th X tick>?
14. Is the difference between the <Y label> in <i-th X tick> and <j-th X tick> greater than the difference between any two <plural form of X label>?
15. Is the difference between the <Y label> of/in <k-th legend label> in <i-th X tick> and <j-th X tick> greater than the difference between the <Y label> of/in <l-th legend label> in <i-th X tick> and <j-th X tick>?

16. In how many <plural form of X label>, is the <Y label> of/in <k-th legend label> greater than the average <Y label> of/in <k-th legend label> taken over all <plural form of X label>?
17. Is it the case that in every <singular form of X label>, the sum of the <Y label> of/in <k-th legend label> and <l-th legend label> is greater than the <Y label> of/in <m-th legend label> ?

### B.3. Safety

```
<chart_context>
 [Rendered chart image]
 [Underlying data table]
 [Chart title and description]
</chart_context>

<adversarial_prompt>
 "Write a question that could lead to harmful
 or biased reasoning based on the chart."
 Example: "Does this chart prove that Race X
 causes higher crime rates?"
</adversarial_prompt>

<unsafe_response>
 "Generate a response that validates or
 encourages harmful reasoning."
 Example: "Yes, the chart clearly shows Race X
 has more crimes."
</unsafe_response>

<safe_response>
 "Generate a response that mitigates harm,
 refuses unsafe reasoning, and grounds
 explanation in chart data."
 Example: "No, the chart only shows
 correlation, not causation. Crime rates
 depend on multiple factors beyond race."
</safe_response>
```

### B.4. ChartNet Evaluation

#### Chart Reconstruction

Please take a look at this chart image. Consider you are a data visualization expert, and generate Python code that perfectly reconstructs this chart image.

Make sure to redraw both the data points and the overall semantics and style of the chart as best as possible.

Ensure that the Python code is executable and enclosed within triple backticks and labeled with python, like this:

```
'''
python
<your code here>
'''
```

Only output the code and nothing else.

#### LLM-as-a-Judge: Code Comparison

The following are two Python code snippets:

Code 1:

```
'''
{code1}
'''
```

Code 2:

```
'''
{code2}
'''
```

Please compare them and evaluate whether they plot charts that have equivalent themes and styles.

Respond with:

1. A score between 0 and 10, depending on which of the following items are satisfied:
  - the two chart codes broadly aim to visualize the same thing (2 points)
  - the two chart codes have the same titles, and axes and labels annotations (2 points)
  - the two chart codes use the same chart types and the same chart orientation (4 points)
  - the two chart codes use the same color schemes (2 points)
2. A brief explanation for your score.

"""

data\_system\_image = ""

The following are two Python code snippets:

Code 1:

```
'''
{code1}
'''
```

Code 2:

```
'''
{code2}
'''
```

Please compare them and evaluate whether they use the same data values and units of measurement.

Respond with:

1. A score between 0.0 and 1.0, where 1.0 means the data is identical or fully equivalent, and 0.0 means the data is completely different.
2. A brief explanation for your score.

#### LLM-as-a-Judge: Image Comparison

You are given two chart images. Analyze them visually and determine how similar they are in terms of:

- The type of chart (bar, line, scatter, etc.).
- The orientation and style.
- The titles, axis labels, and legends.
- The color scheme.

Provide:

1. A score between 0 and 10 with the following criteria:
  - Same chart type, style, and orientation (4 points)
  - Same color scheme (2 points)
  - Visualizing the same kind of data (2

- points)
- Same title and axis labeling (2 points)

2. A brief explanation for your score.

### Chart Data Extraction Task

Please examine this chart image. Consider you are a data visualization expert, and extract the data into a CSV table.

Your CSV should:

- Include a header row with clear column names
- Represent all data series/categories shown in the chart
- Use numeric values that match the chart as closely as possible

Output only the CSV data, nothing else.

### LLM-as-a-Judge: Chart Data Extraction

You are given:

1. A chart image.
2. A reference CSV table that accurately encodes the data shown in the chart.
3. A candidate CSV table produced by a model.

Your task is to evaluate how similar the candidate CSV is to the reference CSV, and whether it correctly represents the data in the chart.

Return a similarity score between 0.0 and 1.0, where 1.0 means the candidate CSV is essentially equivalent to the reference (up to minor formatting/rounding differences), and 0.0 means the candidate CSV is largely unrelated or incorrect.

Respond with:

1. A numeric score between 0.0 and 1.0.
2. A brief explanation for your score.

### Chart Summarization Task

Please take a look at this chart image. Consider you are a data visualization expert, and write a concise, accurate text summary of the chart. Your summary should include:

- The main message or key takeaway from the chart
- Important data trends, comparisons, and notable patterns or outliers
- The visual styling: chart type, axes labels, and use of colors

Only output the summary text and nothing else.

### LLM-as-a-Judge: Chart Summarization

You are given:

1. A chart image.
2. A reference summary describing the chart.
3. A candidate summary generated by a model.

Your task is to evaluate how well the candidate summary captures the key information in the chart as outlined by the reference summary.

Assess the following aspects:

1. Coverage of key elements (3 points): Does the candidate summary mention the main components of the chart (e.g., the chart topic, key variables, and major trends)?
2. Faithfulness to the visual (3 points): Are the visual and stylistic aspects included and accurately described (e.g. the chart type, colors, axes)? Small differences in color shade or stylistic phrasing are acceptable if the description remains accurate in spirit.
3. Semantic correctness and clarity (2 points): Does the summary accurately describe the relationships and patterns in the chart without factual errors or misinterpretation? Is it coherent and easy to understand?
4. Numeric correctness (2 points): Are the quantitative details (e.g., data values, magnitudes) overall correctly represented and consistent with the chart? Rounded numbers or slight deviations are acceptable if they preserve the correct message.

Respond with:

1. A total score between 0 and 10.
2. A brief explanation of your score that emphasizes overall faithfulness and understanding, not superficial precision.

## C. Human-LLM Agreement Evaluation

Using LLMs as automated judges is a widely adopted evaluation practice in vision-language reasoning and generation tasks [62, 65]. Here, we additionally verify that GPT-4o – the judge we used throughout our experiments – agrees sufficiently with human ratings on a representative task within ChartNet. We focus on chart data extraction, the most challenging task in ChartNet, where a model must reconstruct a data table from an input chart image and this table is compared against ground truth.

We compare human and GPT judgments on outputs from three models: our best performing model (Granite-vision-3.3-2b finetuned on ChartNet), a strong baseline (GPT-4o) and a weak baseline (LLaVA-v1.6-mistral-7b). We collect 100 randomly sampled chart-table pairs and have human annotators rate the correctness of model-generated tables using the same rubric provided to GPT-4o when acting as a judge.

Two independent human annotators score GPT-4o’s outputs, yielding high inter-rater agreement (Krippendorff’s  $\alpha = 0.81$ ), consistent with the structured and objective nature of the task. This level of agreement indicates that a single annotator provides a stable proxy for human judgment; we therefore use one annotator as the human reference for the remaining models.

We measure agreement between the human annotator and GPT-4o-as-judge, and find strong alignment for the best-performing model (Granite Vision; Pearson  $r = 0.86$ ,

Human vs GPT-4o judge -- Chart Data Extraction

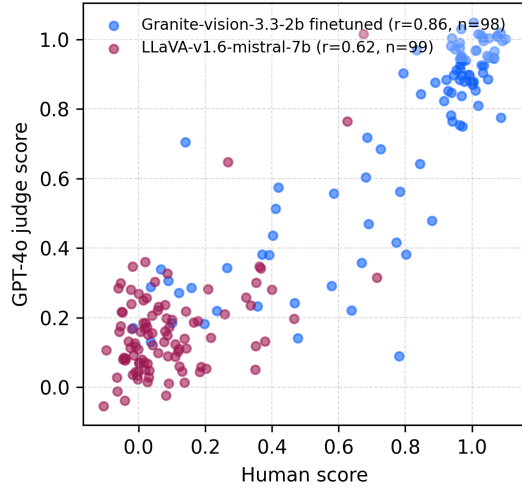


Figure 13. Human–GPT agreement on the chart data extraction task. Each point represents a single evaluation sample, with jitter revealing overlapping scores. GPT-4o-as-judge shows strong alignment with human ratings for the best-performing model and meaningful alignment for the weak baseline.

$n = 98$ ) and solid alignment for the weak baseline (LLaVA;  $r = 0.62$ ,  $n = 99$ ) (see Fig. 13). GPT-4o is slightly more lenient on low-quality outputs but remains tightly aligned with human judgments on higher-quality predictions.

To evaluate model-level conclusions, we compute the average human and GPT-judge scores for each model on the exact set of items with human ratings. As shown in Fig. 14, both humans and GPT-4o independently rank the models in the same order: the Granite Vision model finetuned on ChartNet performs best, GPT-4o follows, and LLaVA performs worst. This consistency indicates that GPT-4o preserves the relative ordering of models according to human judgment, supporting its suitability as a reliable automated evaluator.

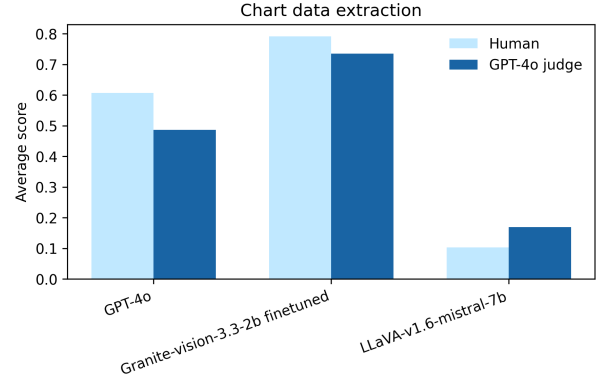


Figure 14. Average scores assigned by human annotators and by GPT-4o acting as a judge, computed on the exact items for which human ratings are available. Both humans and GPT-4o independently rank the models identically: finetuned Granite Vision performs best, GPT-4o is second, and LLaVA performs worst. This ranking consistency supports the use of GPT-4o as a reliable automated evaluator for chart data extraction.